your `<link>` tags to point at files in `/static/`, or you can define redirects that point to your own routes.

Django comes with a series of views as classes, which are handy utilities to quickly serve a file or perform redirects. The `RedirectView` class found in `django.views.generic.base` can be used directly inside your urls.py file, specifying the destination of a redirected URL. The following code redirects a request for /favicon.ico, sending it to /static/img/favicon.ico:

```
from django.urls import path
from django.views.generic.base import RedirectView

urlpatterns = [
    path('favicon.ico',
         RedirectView.as_view(url='/static/img/favicon.ico')),
    # ...
]
```

HTML 5 recommends you provide multiple sizes for your favicon, so it can be used in different resolutions in different contexts. This can lead to requiring half a dozen `<link>` tags in your HTML to get all the possible choices. The RealFaviconGenerator website is a free tool that handles all the cases and provides helpful instructions on what you'll need: https://realfavicongenerator.net/.

13.4 Feature flags

Over time, your Django project may become a product: a site your users are deeply invested in. With this fun comes feature requests. Every code change you make to your site comes with the risk of introducing problems. Of course, like you learned in chapter 8, you should have unit tests, which can help prevent this problem. Even with a good test suite in place, you can still make bad design decisions or misunderstand a requirement. One modern development practice that limits your risk is *feature flags*. The idea is to wrap new code in a conditional block and then allow only certain users to use that code. This technique means you can have users beta test a feature in your production code: only those with the feature flag turned on get to see it.

The cleverly named Django Waffle (https://pypi.org/project/django-waffle/) is a feature flag library. Waffle provides three controls: switches, samples, and flags. A switch is an on–off configuration value; a sample is a control for a percentage of visitors; and a flag goes the whole way, offering fine-grained control over who sees a feature.

You install Django Waffle with the usual `python -m pip install django-waffle`. It requires `"waffle"` in `INSTALLED_APPS` and needs `"waffle.middleware.WaffleMiddleware"` added to the `MIDDLEWARE` configuration. Waffle has its own database tables, which it uses to track state, so you need to run the `migrate` command after installation.

Waffle's simplest control is the switch, which it provides as a `Model`. You can use either the Django Admin or a command management tool to create switches and turn them on or off. Within your code, you query a switch with the `switch_is_active()` function, passing it the name of your `Switch` instance. The function returns a Boolean value, which you use to evaluate your conditional feature block.